

Procesamiento Masivo de Datos: **OpenMP**

Alex Di Genova

Universidad de O'Higgins

OpenMP

Introducción y funciones

Memoria compartida · Regiones paralelas · Trabajo compartido

OpenMP

Introducción

- API para programar sistemas de **memoria compartida** en C, C++ y Fortran.

Sintaxis de una región paralela

```
#pragma omp parallel [clausulas]
{
    /* Bloque estructurado */
}
```

Los corchetes indican elementos opcionales; no son parte del código C.

El programador decide:

OpenMP

Introducción

- API para programar sistemas de **memoria compartida** en C, C++ y Fortran.
- Combina directivas del compilador, funciones de tiempo de ejecución y variables de entorno.

Sintaxis de una región paralela

```
#pragma omp parallel [clausulas]
{
    /* Bloque estructurado */
}
```

Los corchetes indican elementos opcionales; no son parte del código C.

El programador decide:

OpenMP

Introducción

- API para programar sistemas de **memoria compartida** en C, C++ y Fortran.
- Combina directivas del compilador, funciones de tiempo de ejecución y variables de entorno.
- El programador identifica el paralelismo e inserta las directivas apropiadas.

Sintaxis de una región paralela

```
#pragma omp parallel [clausulas]
{
    /* Bloque estructurado */
}
```

Los corchetes indican elementos opcionales; no son parte del código C.

El programador decide:

OpenMP

Introducción

- API para programar sistemas de **memoria compartida** en C, C++ y Fortran.
- Combina directivas del compilador, funciones de tiempo de ejecución y variables de entorno.
- El programador identifica el paralelismo e inserta las directivas apropiadas.
- En C/C++, las directivas comienzan con `##pragma omp`.

Sintaxis de una región paralela

```
#pragma omp parallel [clausulas]
{
    /* Bloque estructurado */
}
```

Los corchetes indican elementos opcionales; no son parte del código C.

El programador decide:

OpenMP

Introducción

- API para programar sistemas de **memoria compartida** en C, C++ y Fortran.
- Combina directivas del compilador, funciones de tiempo de ejecución y variables de entorno.
- El programador identifica el paralelismo e inserta las directivas apropiadas.
- En C/C++, las directivas comienzan con `##pragma omp`.

Sintaxis de una región paralela

```
#pragma omp parallel [clausulas]
{
    /* Bloque estructurado */
}
```

Los corchetes indican elementos opcionales; no son parte del código C.

El programador decide:

- qué código se ejecuta en paralelo;

OpenMP

Introducción

- API para programar sistemas de **memoria compartida** en C, C++ y Fortran.
- Combina directivas del compilador, funciones de tiempo de ejecución y variables de entorno.
- El programador identifica el paralelismo e inserta las directivas apropiadas.
- En C/C++, las directivas comienzan con `##pragma omp`.

Sintaxis de una región paralela

```
#pragma omp parallel [clausulas]
{
    /* Bloque estructurado */
}
```

Los corchetes indican elementos opcionales; no son parte del código C.

El programador decide:

- qué código se ejecuta en paralelo;
- qué variables son privadas o compartidas;

OpenMP

Introducción

- API para programar sistemas de **memoria compartida** en C, C++ y Fortran.
- Combina directivas del compilador, funciones de tiempo de ejecución y variables de entorno.
- El programador identifica el paralelismo e inserta las directivas apropiadas.
- En C/C++, las directivas comienzan con `##pragma omp`.

Sintaxis de una región paralela

```
#pragma omp parallel [clausulas]
{
    /* Bloque estructurado */
}
```

Los corchetes indican elementos opcionales; no son parte del código C.

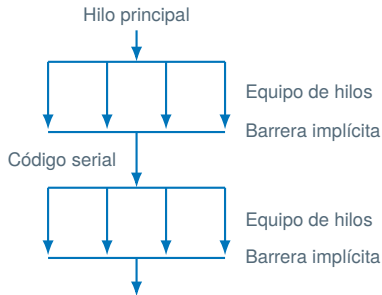
El programador decide:

- qué código se ejecuta en paralelo;
- qué variables son privadas o compartidas;
- cómo sincronizar los accesos.

OpenMP

La región paralela · Modelo fork–join

- La ejecución comienza en un hilo inicial. Una región `parallel` forma un **equipo de hilos**.



Número de hilos solicitado

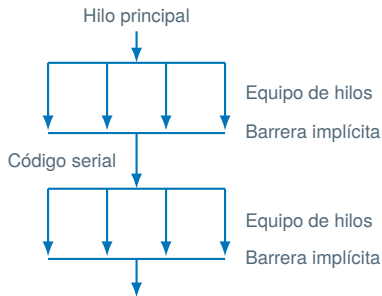
`OMP_NUM_THREADS · omp_set_num_threads()`

Cláusula `num_threads(n)` en una región concreta.

OpenMP

La región paralela · Modelo fork–join

- La ejecución comienza en un hilo inicial. Una región `parallel` forma un **equipo de hilos**.
- El hilo que encuentra la región es el hilo principal del equipo.



Número de hilos solicitado

`OMP_NUM_THREADS · omp_set_num_threads()`

Cláusula `num_threads(n)` en una región concreta.

OpenMP

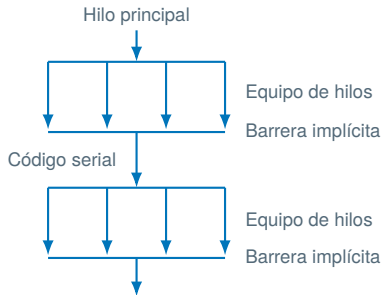
La región paralela · Modelo fork–join

- La ejecución comienza en un hilo inicial. Una región `parallel` forma un **equipo de hilos**.
- El hilo que encuentra la región es el hilo principal del equipo.
- Todos ejecutan el bloque. Las construcciones de trabajo compartido distribuyen tareas.

Número de hilos solicitado

`OMP_NUM_THREADS · omp_set_num_threads()`

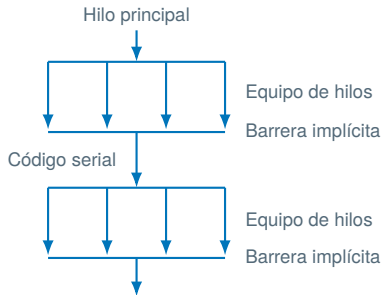
Cláusula `num_threads(n)` en una región concreta.



OpenMP

La región paralela · Modelo fork–join

- La ejecución comienza en un hilo inicial. Una región `parallel` forma un **equipo de hilos**.
- El hilo que encuentra la región es el hilo principal del equipo.
- Todos ejecutan el bloque. Las construcciones de trabajo compartido distribuyen tareas.
- Al terminar la región hay una **barrera implícita**; continúa el hilo que la encontró.



Número de hilos solicitado

`OMP_NUM_THREADS · omp_set_num_threads()`

Cláusula `num_threads(n)` en una región concreta.

OpenMP

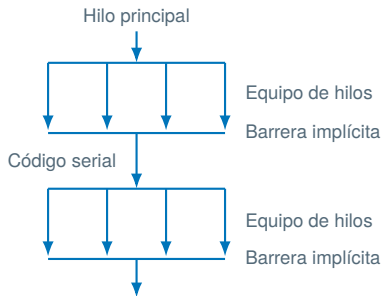
La región paralela · Modelo fork–join

- La ejecución comienza en un hilo inicial. Una región `parallel` forma un **equipo de hilos**.
- El hilo que encuentra la región es el hilo principal del equipo.
- Todos ejecutan el bloque. Las construcciones de trabajo compartido distribuyen tareas.
- Al terminar la región hay una **barrera implícita**;
- Evitar regiones pequeñas ayuda a reducir el sobrecosto.

Número de hilos solicitado

`OMP_NUM_THREADS · omp_set_num_threads()`

Cláusula `num_threads(n)` en una región concreta.



OpenMP

Modelo de memoria

Variables privadas

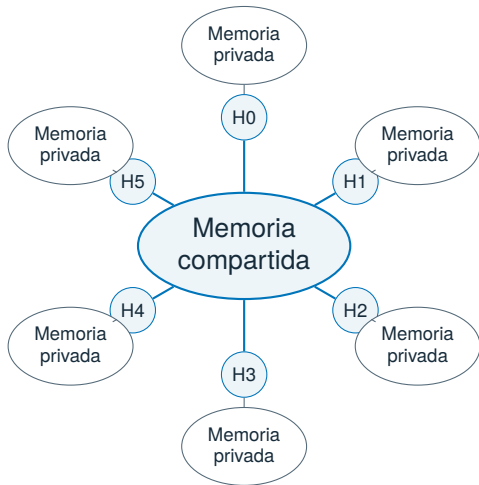
Cada hilo dispone de su propia copia. Un mismo nombre puede referirse a ubicaciones distintas de memoria.

Variables compartidas

Los hilos pueden leer y escribir la misma ubicación. El programador debe coordinar los accesos que puedan entrar en conflicto.

Las variables globales se comparten por defecto, salvo excepciones como `threadprivate`.

Privatizar un puntero no privatiza automáticamente los datos a los que apunta.



OpenMP

Construcciones de trabajo compartido

Dentro de una región paralela, estas construcciones distribuyen trabajo entre los hilos del equipo.

for: iteraciones

Reparte las iteraciones de un ciclo entre los hilos.

```
#pragma omp for
for (int i = 0; i < n;
     i++) {
    procesar(i);
}
```

sections: bloques

Cada sección se ejecuta una vez, por uno de los hilos.

```
#pragma omp sections
{
    #pragma omp section
    { funcion_a(); }
    #pragma omp section
    { funcion_b(); }
}
```

single: un hilo

Un solo hilo ejecuta el bloque; no se especifica cuál.

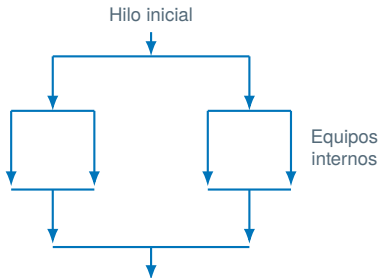
```
#pragma omp single
{
    preparar_datos();
}
```

Por defecto hay una barrera al final de estas construcciones. La cláusula `nowait` permite omitirla cuando sea correcto. Fragmentos ilustrativos.

OpenMP

Paralelismo anidado

Una región paralela puede aparecer dentro de otra. Cada hilo que la encuentra forma un nuevo equipo.



Esquema: 2 hilos externos y 2 hilos por equipo interno, si la configuración permite ambos niveles activos.

Ejemplo del original: 1×5

```
#pragma omp parallel num_threads(1)
{
    Work1();
    #pragma omp parallel num_threads(5)
    {
        Work2();
    }
}
```

La región externa tiene un hilo; la interna solicita cinco.

Niveles activos

Para dos niveles con varios hilos: `omp_set_max_active_levels(2)`. El entorno puede limitar el tamaño de los equipos.

OpenMP

Sincronización

Barrera

Todos los hilos del equipo esperan hasta que todos hayan llegado al mismo punto.

```
producir();  
#pragma omp barrier  
consumir();
```

Debe ser alcanzada por todos los hilos del equipo.

Región crítica

Solo un hilo a la vez ejecuta una región crítica con el mismo nombre.

```
#pragma omp critical  
{  
    sum += c;  
    contador++;  
}
```

Protege un bloque de instrucciones. Las regiones sin nombre comparten la misma exclusión.

Operación atómica

Protege una operación admitida sobre una ubicación concreta de memoria.

```
#pragma omp atomic update  
sum += c;
```

No protege un bloque arbitrario de código.

`critical` y `atomic` no son barreras. Los accesos concurrentes a un dato compartido deben usar una sincronización compatible.

OpenMP

Funciones de tiempo de ejecución

Permiten consultar y modificar la configuración. Por ejemplo, `OMP_NUM_THREADS` fija una solicitud inicial y `omp_set_num_threads()` la cambia para regiones posteriores del contexto de la tarea que llama.

Función	Descripción
<code>omp_set_num_threads(n)</code>	Solicita el número de hilos para regiones posteriores.
<code>omp_get_num_threads()</code>	Número de hilos del equipo actual; fuera de él, 1.
<code>omp_get_max_threads()</code>	Cota superior para un nuevo equipo sin cláusula <code>num_threads</code> .
<code>omp_get_num_procs()</code>	Número de procesadores disponibles para el dispositivo.
<code>omp_get_thread_num()</code>	Identificador del hilo: desde 0 hasta el tamaño del equipo menos 1.
<code>omp_in_parallel()</code>	Indica si se ejecuta dentro de una región paralela activa.
<code>omp_get_dynamic()</code>	Consulta si el ajuste dinámico del número de hilos está habilitado.
<code>omp_set_dynamic(valor)</code>	Habilita o deshabilita ese ajuste.
<code>omp_get_nested()</code>	Consulta el paralelismo anidado; función obsoleta.
<code>omp_set_nested(valor)</code>	Configura el paralelismo anidado; función obsoleta.

Se conservan las dos últimas funciones del original. Alternativas: `omp_get_max_active_levels()` y `omp_set_max_active_levels(n)`.

Ejemplos OpenMP

Hello world

```
#include <omp.h>
#include <stdio.h>
int main(void) {
    #pragma omp parallel
    {
        printf("Hola Mundo... desde hilo = %d\n",
               omp_get_thread_num());
    }
    return 0;
}
```

Compilar y ejecutar en una terminal

```
gcc -fopenmp examples/hello.c -o hello
export OMP_NUM_THREADS=3
./hello
```

Un saludo por hilo

Cada hilo ejecuta el mismo bloque y consulta su identificador con `omp_get_thread_num()`.

Una salida posible

```
Hola Mundo... desde hilo = 1
Hola Mundo... desde hilo = 0
Hola Mundo... desde hilo = 2
```

El orden de los mensajes puede cambiar entre ejecuciones.

En Jupyter/Colab: `%env OMP_NUM_THREADS=3;` anteponer ! a los comandos de compilación y ejecución.

Ejemplos OpenMP

Un for dentro de parallel: trabajo repetido

```
#include <omp.h>
#include <stdio.h>
int main(void) {
    #pragma omp parallel
    {
        for (int k = 0; k < 10; k++) {
            printf("Itr: %d tid=%d\n", k,
                omp_get_thread_num());
        }
    }
    return 0;
}
```

```
gcc -fopenmp examples/for_replicado.c -o for1
OMP_NUM_THREADS=3 ./for1
```

Cada hilo recorre todo el ciclo

Con un equipo de 3 hilos, cada hilo ejecuta 10 iteraciones: **30 mensajes en total.**

H0

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

H1

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

H2

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

Declarar `k` dentro del bloque evita que los hilos compartan el índice y compitan al modificarlo. `parallel` por sí solo no reparte el ciclo.

Ejemplos OpenMP

omp for: distribuir las iteraciones

```
#include <omp.h>
#include <stdio.h>
int main(void) {
    #pragma omp parallel
    {
        #pragma omp for schedule(static)
        for (int k = 0; k < 10; k++) {
            printf("Iter: %d tid=%d\n", k,
                omp_get_thread_num());
        }
    }
    return 0;
}
```

```
gcc -fopenmp examples/for_repartido.c -o for2
OMP_NUM_THREADS=2 ./for2
```

Cada iteración se ejecuta una vez

Con 2 hilos y `schedule(static)`, las 10 iteraciones se reparten en dos bloques de 5.

H0

0	1	2	3	4
---	---	---	---	---

H1

5	6	7	8	9
---	---	---	---	---

El índice del ciclo de `omp for` es privado.
Hay una barrera implícita al final del ciclo.

También se puede escribir `##pragma omp parallel for`. El orden de impresión sigue siendo no determinista.

Ejemplos OpenMP

Funciones de trabajo para sections, single y anidamiento

Las funciones de `work.h` imprimen el trabajo y el identificador del hilo.

```
#include <omp.h>
#include <stdio.h>
#include <unistd.h>
```

```
static inline void Work1(void) {
    printf("executing work 1 hilo: %d\n",
        omp_get_thread_num());
    sleep(1);
}
static inline void Work2(void) {
    printf("executing work 2 hilo: %d\n",
        omp_get_thread_num());
    sleep(1);
}
```

```
static inline void Work3(void) {
    printf("executing work 3 hilo: %d\n",
        omp_get_thread_num());
}
static inline void Work4(void) {
    printf("executing work 4 hilo: %d\n",
        omp_get_thread_num());
    sleep(1);
}
```

Simular trabajo

`sleep(1)` espera un segundo en `Work1`, `Work2` y `Work4`.

Archivo auxiliar compartido por los siguientes ejemplos. `unistd.h` y `sleep` requieren un entorno POSIX, como Linux o Colab.

Ejemplos OpenMP

Secciones paralelas

```
#include "work.h"
int main(void) {
    #pragma omp parallel sections
    {
        #pragma omp section
        { Work1(); }
        #pragma omp section
        {
            Work2();
            Work3();
        }
        #pragma omp section
        { Work4(); }
    }
    return 0;
}
```

- Tres secciones para cinco hilos: algunos no reciben trabajo.

```
gcc -fopenmp examples/sections.c -o
sections
OMP_NUM_THREADS=5 ./sections
```

Una salida posible

```
executing work 1 hilo:1
executing work 2 hilo:2
executing work 4 hilo:3
executing work 3 hilo:2
```

El reparto de secciones entre hilos puede variar.

Ejemplos OpenMP

Secciones paralelas

```
#include "work.h"
int main(void) {
    #pragma omp parallel sections
    {
        #pragma omp section
        { Work1(); }
        #pragma omp section
        {
            Work2();
            Work3();
        }
        #pragma omp section
        { Work4(); }
    }
    return 0;
}
```

- Tres secciones para cinco hilos: algunos no reciben trabajo.
- Work2() y Work3() pertenecen a la misma sección: se ejecutan en orden y en el mismo hilo.

```
gcc -fopenmp examples/sections.c -o
sections
OMP_NUM_THREADS=5 ./sections
```

Una salida posible

```
executing work 1 hilo:1
executing work 2 hilo:2
executing work 4 hilo:3
executing work 3 hilo:2
```

El reparto de secciones entre hilos puede variar.

Ejemplos OpenMP

single: un bloque ejecutado por un hilo

```
#include "work.h"
int main(void) {
    #pragma omp parallel
    {
        Work1();
        #pragma omp single
        {
            Work2();
            Work3();
        }
        Work4();
    }
    return 0;
}
```

```
gcc -fopenmp examples/single.c -o single
OMP_NUM_THREADS=10 ./single
```

Con un equipo de 10 hilos

Work1() 10 ejecuciones: una por hilo.

Work2() 1 ejecución.

Work3() 1 ejecución, en el mismo hilo que Work2().

Work4() 10 ejecuciones, después de la barrera de single.

El hilo que ejecuta `single` no tiene por qué ser el hilo principal.

No hay barrera de entrada: `Work2()` puede comenzar mientras otros hilos aún ejecutan `Work1()`. Todos esperan al final de `single`.

Ejemplos OpenMP

Regiones paralelas anidadas

```
#include "work.h"
int main(void) {
    #pragma omp parallel num_threads(1)
    {
        Work1();
        #pragma omp parallel num_threads(5)
        {
            Work2();
        }
    }
    return 0;
}
```

```
gcc -fopenmp examples/nested.c -o nested
OMP_DYNAMIC=FALSE ./nested
```

Una salida posible

```
executing work 1 hilo:0
executing work 2 hilo:2
executing work 2 hilo:4
executing work 2 hilo:0
executing work 2 hilo:1
executing work 2 hilo:3
```

- El equipo externo tiene un único hilo.

La región externa de un hilo es inactiva: este caso solo necesita un nivel activo.

Ejemplos OpenMP

Regiones paralelas anidadas

```
#include "work.h"
int main(void) {
    #pragma omp parallel num_threads(1)
    {
        Work1();
        #pragma omp parallel num_threads(5)
        {
            Work2();
        }
    }
    return 0;
}
```

```
gcc -fopenmp examples/nested.c -o nested
OMP_DYNAMIC=FALSE ./nested
```

Una salida posible

```
executing work 1 hilo:0
executing work 2 hilo:2
executing work 2 hilo:4
executing work 2 hilo:0
executing work 2 hilo:1
executing work 2 hilo:3
```

- El equipo externo tiene un único hilo.
- La región interna solicita un equipo de cinco hilos.

La región externa de un hilo es inactiva: este caso solo necesita un nivel activo.

Ejemplos OpenMP

Regiones paralelas anidadas

```
#include "work.h"
int main(void) {
    #pragma omp parallel num_threads(1)
    {
        Work1();
        #pragma omp parallel num_threads(5)
        {
            Work2();
        }
    }
    return 0;
}
```

```
gcc -fopenmp examples/nested.c -o nested
OMP_DYNAMIC=FALSE ./nested
```

Una salida posible

```
executing work 1 hilo:0
executing work 2 hilo:2
executing work 2 hilo:4
executing work 2 hilo:0
executing work 2 hilo:1
executing work 2 hilo:3
```

- El equipo externo tiene un único hilo.
- La región interna solicita un equipo de cinco hilos.
- Los identificadores son locales a cada equipo.

La región externa de un hilo es inactiva: este caso solo necesita un nivel activo.

Ejemplos OpenMP

Acumulación compartida con una sección crítica

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int sum = 0, contribution[10];
    /* Generar los valores en serie. */
    for (int k = 0; k < 10; k++)
        contribution[k] = rand() % 50;
    #pragma omp parallel for shared(sum, contribution)
    for (int k = 0; k < 10; k++) {
        int c = contribution[k];
        printf("Itr: %d tid=%d, my_contri=%d\n",
            k, omp_get_thread_num(), c);
        #pragma omp critical
        { sum += c; }
    }
    printf("Sum=%d\n", sum);
    return 0;
}
```

Proteger la actualización

`sum += c` lee y escribe la variable compartida. `critical` permite una actualización a la vez.

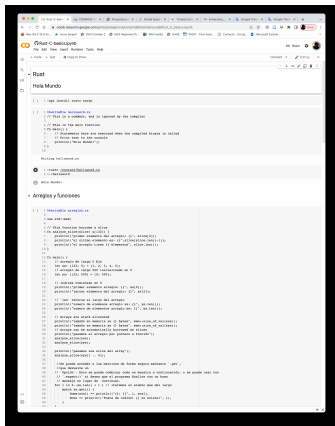
```
gcc -fopenmp \
    examples/critical.c -o
    critical
OMP_NUM_THREADS=5 ./critical
```

Cada iteración imprime su contribución; al salir del ciclo se muestra la suma total.

`rand()` se llama en serie. Sus valores dependen de la implementación de C.

Google Colab

Material práctico de la clase



Notebook de OpenMP

Ejemplos de compilación y ejecución en un notebook.

► Abrir OpenMP.ipynb en GitHub

<https://github.com/adigenova/uohpmd/blob/main/code/OpenMP.ipynb>

Enlace y captura conservados del PDF original.

¿Consultas?

¿Consultas o comentarios?

Muchas gracias